

AI Investment Advisor

1. Project Summary

A full Streamlit application with sign-up and login (SQLAlchemy + PBKDF2). After signing in, the user enters a stock ticker and, with one click, triggers a complete analysis pipeline: live market and news data, native TradingView-aligned technical indicators, a deterministic scoring engine (0–100), and a GPT-4.1 narrative that explains the score without deciding it. Every analysis is saved to the user's history. This is an educational tool only and not financial advice.

2. Goals and Requirements

- Frontend in Streamlit (multi-page: landing / auth / dashboard) and Backend in Python.
- A user authentication layer (SQLAlchemy) with saved analysis history.
- A connection to Finnhub for live price and news, and to yfinance for OHLCV history.
- A deterministic, transparent scoring engine combining technical indicators with AI sentiment (FinBERT).
- GPT-4.1 narrates an explanation based on the score; a floating chatbot grounded in live data.

3. Architecture

The system is layered: Auth → Data → Analysis → Scoring → AI (narrative + chatbot) → Presentation (Streamlit). Guiding principle: the deterministic scoring engine decides, and the LLM only explains — so the same data always produces the same recommendation, and the AI can never bias the rating itself.

Output: dashboard + GPT narrative + grounded chatbot + saved to history
AI layer — GPT-4.1 narrates the score (does not decide it)
Scoring engine — Trend 35% · Momentum 25% · Volume 15% · Volatility 10% · Sentiment 10% · Risk 5%
Analysis layer — native indicators (RSI/MACD/EMA/Bollinger/ADX/ATR/OBV) + sentiment (FinBERT)
Data layer — Finnhub (price/news) + yfinance (history)
Auth layer — sign-up/login (SQLAlchemy) · signed session token

4. Technology Stack

Technology	Component
Streamlit (multi-page)	User interface (Frontend)
Python · pandas · numpy	Backend and computation
SQLAlchemy · PBKDF2	User authentication and history
SQLite / Postgres	Database (DATABASE_URL)
Finnhub	Live price and financial news
yfinance	OHLCV history (2 years)
FinBERT / VADER	Sentiment analysis (AI)
OpenAI GPT-4.1	Narrative explanation of the rating
Groq (Llama 3.3) / Gemini	Grounded floating chatbot
TradingView Lightweight Charts	Interactive charts
Streamlit Community Cloud · Docker	Deployment

5. How It Works — The Data Pipeline

- The user logs in or registers, enters a ticker, and clicks Analyze.
- Live price from Finnhub + two years of OHLCV history from yfinance; up to 10 recent headlines from Finnhub.

- The indicators module computes RSI/MACD/EMA/Bollinger/StochRSI/ADX/ATR/OBV; sentiment is scored with FinBERT.
- correlation.py de-duplicates correlated trend indicators before scoring; see Section 6.1.
- scoring.py produces a 0–100 score and a confidence level (distance from 50 plus agreement across categories).
- GPT-4.1 narrates a plain-language rationale based on the score; the result is saved to the user's history.

6. Indicators and the Scoring Engine

Every indicator is implemented natively in NumPy/pandas (not the pandas-ta library), using the conventions that match TradingView: Wilder's RMA for RSI/ATR/ADX, EMA for MACD, and population standard deviation for Bollinger Bands. The final score blends five active categories — Trend 35% · Momentum 25% · Volume 15% · Volatility 10% · Sentiment 10% (as defined in config.py) — where every indicator exposes a value, a signal and its contribution to the category average, so nothing is a black box. Note: config.py also defines a 5% Risk Adjustment weight, but there is currently no active Risk category in scoring.py — that weight is defined but not yet implemented.

6.1 Diagnosing a Systematic Bias

Testing against TradingView found that the original engine consistently leaned toward 'Strong Buy' even when other sites showed Sell or Neutral. The diagnosis: six or seven correlated trend indicators (price vs. SMA50/200, moving-average crossovers, and so on) were counted as independent signals, so a single phenomenon — an uptrend — was counted almost six times over. The solution: correlation.py, which automatically detects correlated indicators (above a 0.85 threshold) and merges them into a shared cluster weight — now wired directly into scoring.py's Trend category, with ADX kept as an independent signal (trend_adx_share) since it measures trend strength rather than direction. Verified with a synthetic test: a scenario with 5 correlated bullish signals against 2 independent bearish ones scored 71.4/100 (Strong Buy) under the old equal-weighting, and correctly 35.0/100 (mixed/bearish) after the fix.

7. Robustness and Testing

The scoring engine does not depend on API keys — it runs in full even without OpenAI (only the narrative is disabled); sentiment falls back from FinBERT to VADER under constrained memory; and the most recent candle is automatically dropped if it is still forming, to prevent look-ahead bias. Missing data is shown as 'Data unavailable' and is never fabricated. A pytest suite (tests/) verifies closed-form values for every indicator, the absence of look-ahead bias, determinism of the scoring, and sanity against real tickers.

8. Limitations and Future Work

- The 5% Risk Adjustment category is defined in config.py but is not implemented in scoring.py.
- Confidence still depends heavily on the score's distance from 50, not only on agreement between categories.
- Educational tool only; indicators are backward-looking; a high score does not guarantee a return.
- News coverage skews English-language; the default SQLite setup is not meant for high concurrent load.
- Future work: implement the Risk Adjustment category, full historical backtesting.
- Future work: multi-stock portfolios, automated alerts, additional data sources (SEC filings, macroeconomic data).

9. Deployment

The application is deployed and live on Streamlit Community Cloud, built directly from the repository's main branch: ai-investment-advisor-0.streamlit.app

It is also packaged with Docker (a Dockerfile at the project root) and is suitable for deployment on any container-based service (for example, a Hugging Face Spaces Docker Space). In either case, the API keys and DATABASE_URL must be set as environment variables / Secrets in the platform's settings. Full details are in the README.